Linear Algebra in Practice — 80/20 Interview Notes

0. What you actually need for an ML/AI interview

P0 — Master these

1. NumPy arrays
2. Vector operations
3. Element-wise operations / ufuncs
4. sum, max, min
5. Norms
6. Euclidean distance
7. Dot product
8. Broadcasting
9. Matrix representation
10. Matrix shape
11. Matrix multiplication
12. Transpose
13. reshape
14. Matrix × vector
15. Batch dimensions / data shape

P1 — Understand well

- Projection
- Gram–Schmidt
- Row-major storage
- Why NumPy is faster than Python loops
- Numerical problems with nearly dependent vectors
- Batch-first vs batch-last

P2 — Know conceptually

- Custom Matrix implementation
- Detailed memory-index transformation

- Fibonacci matrix identity
- Bilinear forms
- Hadamard product

The uploaded chapter itself emphasizes that NumPy arrays are the practical representation for numerical computation and that matrices are fundamental building blocks of machine learning.

---

1. NumPy Array

Definition

A NumPy ndarray is a multidimensional numerical array designed for efficient numerical computation.

Instead of:

Python List

[1, 2, 3, 4]

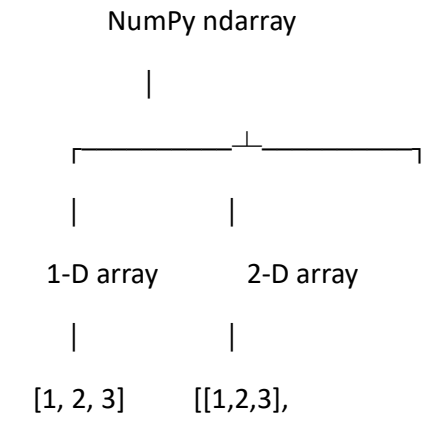
we use:

import numpy as np

x = np.array([1, 2, 3, 4])

The chapter uses NumPy because it is designed for linear algebra and provides much faster operations than ordinary Python objects.

Diagram



[4,5,6]]

| |

vector matrix

Important interview point

A NumPy array is **not automatically a matrix**.

x = np.array([1, 2, 3])

Shape:

(3,)

This is a 1-D array.

Whereas:

x = np.array([[1],

[2],

[3]])

has:

(3, 1)

This is a 2-D column-shaped array.

Interview question

Q: What is the difference between (3,) and (3,1)?

Answer:

(3,) → one-dimensional array

(3,1) → two-dimensional array with 3 rows and 1 column

This distinction becomes extremely important in ML because shape determines whether matrix operations are valid.

---

2. Vectorized Operations

Definition

A vectorized operation applies an operation to an entire NumPy array without explicitly writing a Python loop.

Example:

```
x = np.array([1, 2, 3])


x + 10
```

Result:

```
[11, 12, 13]
```

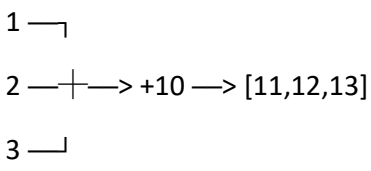
Instead of:

```
result = []

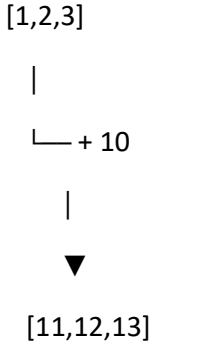

for value in x:
    result.append(value + 10)
```

Diagram

Python loop



Vectorized



The chapter explains that NumPy's operations are implemented through optimized lower-level numerical libraries, allowing vectorized computation to be substantially faster than Python loops.

---

3. NumPy ufuncs

Definition

A **ufunc — universal function** — operates element-by-element on NumPy arrays.

Example:

x = np.array([1, 2, 3])

np.exp(x)

Conceptually:

$[e^1, e^2, e^3]$

Why important?

Instead of:

from math import exp

result = np.array([exp(v) for v in x])

use:

np.exp(x)

The chapter demonstrates that np.exp gives the same result while being dramatically faster; its example measured roughly a **21× speed improvement** for the tested workload.

Interview answer

"Vectorization reduces Python-level looping and allows NumPy to execute numerical operations using optimized native implementations."

That's a much stronger answer than:

"NumPy is faster."

---

4. Aggregation

Aggregation converts multiple values into a smaller result, often a scalar.

Important functions

np.sum(x)

np.prod(x)

np.max(x)

np.min(x)

Example:

x = np.array([1, 2, 3, 4])

np.sum(x)

Output:

10

Mathematically:

$$\sum_{i=1}^{n} x_i$$

The chapter covers np.sum, np.prod, np.max, and np.min as the main array aggregation operations.

---

5. Norm

Definition

A **norm measures the magnitude/length of a vector**.

For the Euclidean norm:

$$\| x \|_2 = \sqrt{\sum_{i=1}^{n} x_i^2}$$

Example:

x = [3, 4]

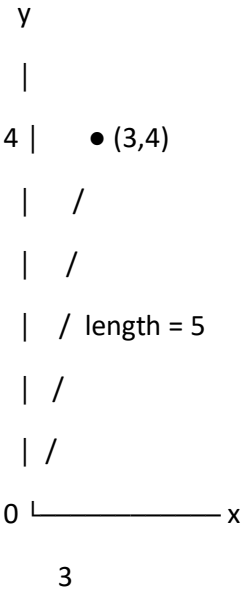
Then:

$\sqrt{(3^2 + 4^2)}$

$= \sqrt{25}$

$= 5$

Diagram



Implementation

From scratch:

```
def euclidean_norm(x):
   return np.sqrt(np.sum(x ** 2))
```

Practical NumPy:

```
np.linalg.norm(x)
```

The chapter gives exactly this progression from the direct implementation to np.linalg.norm.

Interview question

Q: Why use np.linalg.norm() instead of implementing it manually?

Because the library implementation is concise, tested, optimized, and supports different norm orders.

---

6. p-Norm

The general p-norm is:

$$\boxed{\| x \|_p = \left(\sum_i | x_i |^p \right)^{1/p}}$$

For infinity:

$$\boxed{\| x \|_\infty = \max_i | x_i |}$$

Important cases

L1 norm

$$\| x \|_1 = \sum_i | x_i |$$

Example:

x = [-3, 4]

L1 = 3 + 4 = 7

L2 norm

$$\| x \|_2 = \sqrt{\sum_i x_i^2}$$

L∞ norm

maximum absolute value

The chapter implements the general p-norm and then points out that np.linalg.norm(..., ord=p) provides the functionality directly.

Interview connection

Norms appear everywhere in ML:

Regularization

```
    |
    ├── L1
    |
    └── L2
```

Distance

```
    |
    └── Euclidean distance
```

Embeddings

```
    |
    └── normalization
```

---

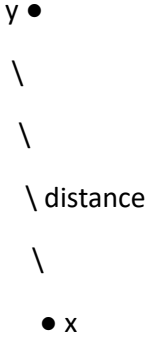
7. Euclidean Distance

Definition

The Euclidean distance between two vectors is:

$$\boxed{d(x,y) = \| x - y \|_2}$$

Diagram



Implementation

```
def euclidean_distance(x, y):
    return np.linalg.norm(x - y, ord=2)
```

The chapter uses precisely this formulation.

ML example

Suppose:

User A = [2, 3]

User B = [5, 7]

Distance:

difference = [-3, -4]

distance = √(9 + 16)

= 5

Smaller distance means the points are geometrically closer under Euclidean distance.

---

8. Dot Product ⭐ ⭐ ⭐

This is **P0**.

Definition

For:

$$x = [x_1, x_2, \dots, x_n]$$

and

$$y = [y_1, y_2, \dots, y_n]$$

the dot product is:

$$x \cdot y = \sum_{i=1}^{n} x_i \, y_i$$

Diagram

$x = [x_1, x_2, x_3]$

$y = [y_1, y_2, y_3]$

|

▼

$x_1y_1 + x_2y_2 + x_3y_3$

|

▼

scalar

Implementation

From scratch:

```
def dot_product(x, y):
    return np.sum(x * y)
```

Practical:

```
np.dot(x, y)
```

The chapter demonstrates both and specifically recommends np.dot because it avoids a subtle broadcasting problem.

---

9. Dot Product — Geometric Meaning

The dot product also satisfies:

$$\boxed{x \cdot y = \| x \| \| y \| \cos\theta}$$

Therefore:

x · y > 0 → angle < 90°

x · y = 0 → perpendicular

x · y < 0 → angle > 90°

Why ML engineers care

Dot products are the foundation of:

- linear regression
- neural-network layers
- attention mechanisms
- cosine similarity
- embeddings
- vector search

The chapter explicitly notes that matrix-vector multiplication forms a fundamental building block for linear regression and fully connected neural-network layers.

---

10. Broadcasting ⭐ ⭐ ⭐

This is a **very important interview topic**.

Definition

Broadcasting allows NumPy to perform operations on arrays with compatible but different shapes by logically expanding dimensions.

Example:

x = np.array([1, 2, 3])

x * 2

Conceptually:

[1,2,3]

×

[2,2,2]

↓

[2,4,6]

Dangerous example

Suppose:

x = np.array([1, 2, 3, 4])

y = np.array([2])

Then:

x * y

becomes:

[1,2,3,4]

×

[2,2,2,2]

↓

[2,4,6,8]

The chapter warns that this can produce code that **runs successfully but gives an incorrect mathematical result**.

Interview question

Q: Why can broadcasting be dangerous?

Excellent answer:

Broadcasting is convenient because NumPy automatically expands compatible dimensions, but it can hide shape mistakes. A program may execute without an exception while performing an operation that is mathematically different from what the developer intended.

That is a **senior-level answer**.

---

11. np.dot() vs x * y

This is a common interview trap.

x * y

means:

element-wise multiplication.

While:

np.dot(x, y)

for 1-D vectors means:

dot product.

Example:

x = [1,2,3]

y = [4,5,6]

Element-wise:

x * y = [4,10,18]

Dot:

1×4 + 2×5 + 3×6

= 32

---

12. Orthogonal Projection

Definition

Projection finds the component of a vector lying in another vector/subspace.

For projection onto vector e:

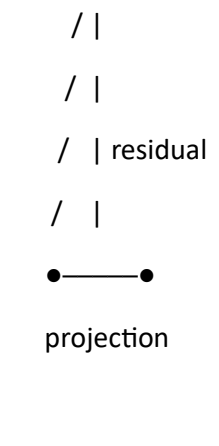
$$\boxed{proj_e(x) = \frac{x \cdot e}{e \cdot e} e}$$

Diagram

x

●

/|



The chapter defines projection onto a subspace as a sum of projections onto its orthogonal basis vectors.

Implementation

```
def projection(x, e):
    return np.dot(x, e) * e / np.dot(e, e)
```

For multiple basis vectors:

```
def projection(x, to):
    p_x = np.zeros_like(x)

    for e in to:
        e_norm_square = np.dot(e, e)
        p_x += np.dot(x, e) * e / e_norm_square

    return p_x
```

Verification

If:

p = projection(x, e)

then:

x - p

should be perpendicular to e.

Check:

np.allclose(np.dot(e, x - p), 0.0)

The source uses exactly this orthogonality check.

---

13. Gram–Schmidt ⭐ ⭐

Definition

Gram–Schmidt converts linearly independent vectors into an **orthonormal set**.

The result satisfies:

$$\| e_i \| = 1$$

and

$$e_i \cdot e_j = 0 \; i \neq j$$

while preserving the relevant spans.

Intuition

Suppose:

v_1

v_2

v_3

We want:

e_1

e_2

e_3

such that:

$e_1 \perp e_2$

$e_1 \perp e_3$

$e_2 \perp e_3$

and

$|e_1| = |e_2| = |e_3| = 1$

Algorithm

Step 1

Take:

$$u_1 = v_1$$

Step 2

Remove the projection of $v_2$onto u_1:

$$u_2 = v_2 - proj_{u_1}(v_2)$$

Step 3

For v_3:

$$u_3 = v_3 - proj_{u_1}(v_3) - proj_{u_2}(v_3)$$

Step 4

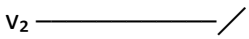
Normalize:

$$e_i = \frac{u_i}{\| u_i \|}$$

Diagram

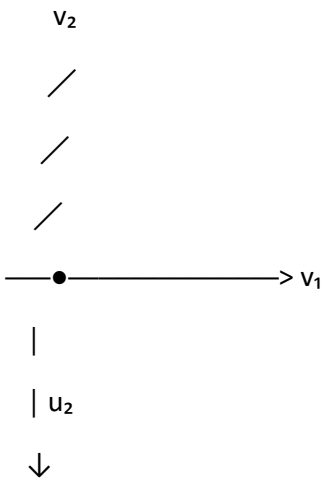
Original vectors

v_1 ——————————>



Remove component of v_2

parallel to v_1



Implementation

```
def gram_schmidt(vectors):
    output = []

    output.append(vectors[0])

    for v in vectors[1:]:
        v_proj = projection(v, output)
        output.append(v - v_proj)

    output = [
        v / np.linalg.norm(v, ord=2)
```

```
    for v in output
]


return output
```

This is the implementation presented in the chapter.

---

14. Gram–Schmidt Production Problem ⭐⭐

Suppose input vectors are linearly dependent.

Example:

$v_2 = 2v_1$

Then after projection:

v_2 - projection(v_2)

≈ 0

Now normalization requires:

v / np.linalg.norm(v)

but:

norm(v) ≈ 0

which creates numerical instability.

The chapter explicitly identifies this problem and notes that very small norms can cause problematic division.

Interview answer

If interviewer asks:

"What can go wrong with Gram–Schmidt?"

Say:

"Linearly dependent or nearly dependent vectors can produce a vector with a very small norm. Normalizing that vector causes numerical instability. In production, I would detect near-zero norms rather than blindly normalize."

That is much better than simply knowing the algorithm.

---

15. Matrix ⭐⭐⭐

Definition

A matrix is a rectangular arrangement of numbers.

$$A \in \mathbb{R}^{n \times m}$$

means:

n rows

m columns

Example:

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$$

has:

2 × 3

The source describes matrices as tables of numbers and highlights their role in representing systems, graphs, mappings, and ML computations.

---

16. Matrix Shape

For:

```
A = np.array([
    [1, 2, 3],
    [4, 5, 6]
])
```

A.shape

returns:

(2, 3)

Meaning:

2 rows

3 columns

Interview rule ⭐ ⭐ ⭐

For:

$$A_{m\times n}B_{n\times p}$$

the middle dimensions must match.

(m × n) × (n × p)

↑

must match

result = m × p

Example:

(2 × 3) × (3 × 4)

↓

valid

result = (2 × 4)

The source explicitly emphasizes that the second dimension of the first matrix must match the first dimension of the second.

---

17. Matrix Addition

Matrices can be added when their shapes match.

$$A + B = [a_{ij} + b_{ij}]$$

Example:

A = [1 2]

[3 4]

B = [5 6]

[7 8]

Then:

A+B

= [6 8]

[10 12]

Implementation:

A + B

---

18. Element-wise Matrix Multiplication ⭐ ⭐ ⭐

This is **not matrix multiplication**.

A * B

means element-wise/Hadamard multiplication.

Example:

A = [1 2]

[3 4]

B = [5 6]

[7 8]

A * B

= [5 12]

[21 32]

The chapter demonstrates that NumPy's * performs pointwise multiplication.

---

19. Matrix Multiplication ⭐ ⭐ ⭐ ⭐ ⭐

This is one of the **highest-value concepts in the chapter**.

Definition

For:

$$A \in \mathbb{R}^{n \times m}$$

and

$$B \in \mathbb{R}^{m \times l}$$

we get:

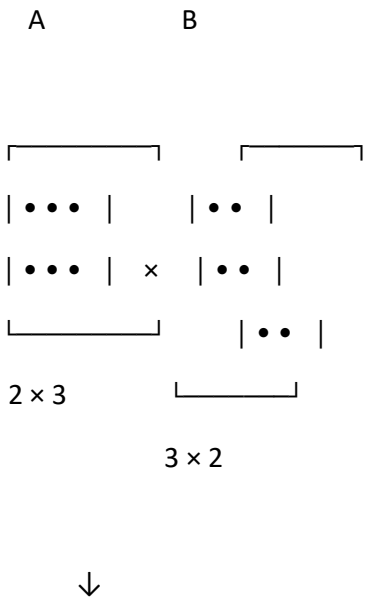
$$AB \in \mathbb{R}^{n \times l}$$

Each output element is:

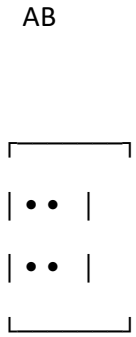
$$\boxed{(AB)_{ij} = \sum_{k} A_{ik} B_{kj}}$$

The source defines matrix multiplication this way and explains that each element is the dot product of a row of A and a column of B.

Diagram ⭐ ⭐ ⭐ ⭐ ⭐



↓



Mental model

For:

A = rows

B = columns

each result cell is:

row of A

  |

  | dot product

  ▼

column of B

  |

  ▼

one cell of AB

---

20. Matrix Multiplication Example

Let:

A = [1 2]

  [3 4]

B = [5 6]

[7 8]

Then:

AB[0,0]

= 1×5 + 2×7

= 19

AB[0,1]

= 1×6 + 2×8

= 22

AB[1,0]

= 3×5 + 4×7

= 43

AB[1,1]

= 3×6 + 4×8

= 50

Therefore:

AB = [19 22]

[43 50]

---

21. Implementation From Scratch

The chapter implements matrix multiplication manually:

```
def matrix_multiplication(A, B):

    if A.shape[1] != B.shape[0]:
```

```
        raise ValueError(
            "The number of columns in A "
            "must match the number of rows in B."
        )

    AB = np.zeros(
        shape=(A.shape[0], B.shape[1])
    )

    for i in range(A.shape[0]):
        for j in range(B.shape[1]):
            AB[i, j] = np.sum(
                A[i, :] * B[:, j]
            )

    return AB
```

The implementation and dimension check are directly shown in the source.

Practical implementation

Don't normally write that yourself.

Use:

A @ B

or:

np.matmul(A, B)

The source demonstrates that @ and np.matmul() produce the same matrix multiplication result.

---

22. * vs @ ⭐ ⭐ ⭐ ⭐ ⭐

Memorize this.

Operation	Meaning
A * B	Element-wise multiplication
A @ B	Matrix multiplication
np.matmul(A,B)	Matrix multiplication
np.dot(A,B)	Dot product / generalized dot behavior
A.T	Transpose

Interview trap

If interviewer asks:

"What's the difference between A * B and A @ B?"

Answer immediately:

"* performs element-wise multiplication, whereas @ performs matrix multiplication."

---

23. Matrix Transpose

Definition

Transpose swaps rows and columns.

$$\boxed{(A^T)_{ij} = A_{ji}}$$

Example:

A = [1 2 3]

[4 5 6]

Transpose:

A^T = [1 4]

[2 5]

[3 6]

Diagram

transpose

|

▼

1 2 3 1 4

4 5 6 —> 2 5

3 6

2×3 3×2

The source defines transpose as replacing rows with columns.

NumPy

A.T

or:

np.transpose(A)

---

24. Important Transpose Identity

$$\boxed{(AB)^T = B^T A^T}$$

Notice the order reverses.

Interview trap

Wrong:

$(AB)^T = A^T B^T$

Correct:

$(AB)^T = B^T A^T$

The source explicitly states this property.

---

25. Matrix Multiplication Is NOT Commutative ⭐ ⭐ ⭐ ⭐ ⭐

For normal numbers:

a × b = b × a

But matrices generally satisfy:

AB ≠ BA

Sometimes:

AB

is valid while:

BA

isn't even defined.

The chapter explicitly emphasizes this distinction.

Interview answer

"Matrix multiplication is associative but generally not commutative."

That one sentence is worth remembering.

---

26. Matrix Multiplication Properties

Associative

$$\boxed{A(BC) = (AB)C}$$

Distributive

$$\boxed{A(B + C) = AB + AC}$$

and

$$\boxed{(A + B)C = AC + BC}$$

Not generally commutative

$$\boxed{AB \neq BA}$$

These properties are explicitly listed in the source.

---

27. Matrix × Vector ⭐⭐⭐⭐⭐

This is extremely important for ML.

Suppose:

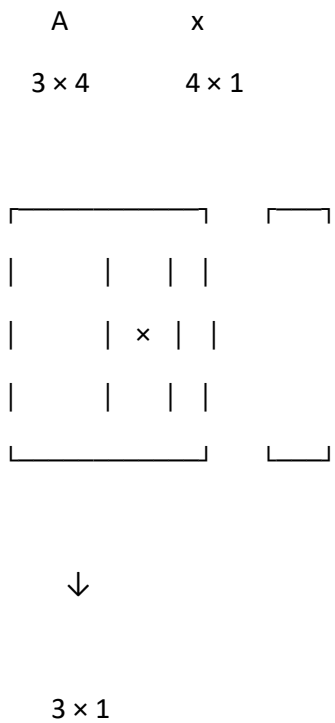
A = 3 × 4

x = 4 × 1

Then:

A x = 3 × 1

Diagram:



The source explains Ax as a transformation of the input vector and connects it directly to linear regression and fully connected neural-network layers.

---

28. Why Ax Matters in Machine Learning

A linear layer is essentially:

$$\boxed{y = Wx + b}$$

The important computational component is:

W @ x

Conceptually:

Input features

|

▼

x

|

|

▼

W

|

▼

W @ x

|

▼

transformed features

This is why understanding matrix multiplication is more important than memorizing dozens of NumPy functions.

---

29. Matrix as a Transformation ⭐ ⭐ ⭐ ⭐ ⭐

One of the most important conceptual ideas.

Instead of thinking:

Matrix = table of numbers

think:

Matrix = transformation

$$\boxed{x \rightarrow Ax}$$

The source explicitly develops this interpretation: the matrix maps input data x to Ax.

Example

Suppose:

A = [2 0]

[0 2]

and:

x = [1]

[2]

Then:

Ax = [2]

[4]

The matrix has scaled the vector by 2.

So:

Matrix

|

└── transformation

|

▼

vector

|

▼

transformed vector

This mental model becomes extremely useful when learning:

- neural networks
- PCA
- eigenvectors
- linear regression
- embeddings

- computer vision
- deep learning

---

30. Matrix Storage in Memory

A computer's memory is fundamentally one-dimensional.

So a matrix:

[1 2 3]

[4 5 6]

[7 8 9]

can conceptually be flattened:

[1 2 3 4 5 6 7 8 9]

This is called **row-major ordering** when rows are stored consecutively.

The chapter explains that NumPy represents matrices using multidimensional arrays while the underlying storage can be viewed as a contiguous linear array.

Index conversion

For an $n \times m$matrix:

$$\boxed{(i,j) \to im + j}$$

for row-major storage.

Example:

3 × 3

[0 1 2]

[3 4 5]

[6 7 8]

Element:

(i,j) = (2,1)

linear index:

2×3 + 1

= 7

So the value is:

7

---

31. Custom Matrix Implementation

The chapter creates a simplified Matrix class to demonstrate how a matrix can be represented internally.

Simplified:

class Matrix:

```
    def __init__(self, shape):
        self.shape = shape
        self.data = [
            0.0 for _ in range(shape[0] * shape[1])
        ]

    def _linear_idx(self, i, j):
        return i * self.shape[1] + j

    def __getitem__(self, key):
        i, j = key
        return self.data[self._linear_idx(i, j)]

    def __setitem__(self, key, value):
        i, j = key
        self.data[self._linear_idx(*key)] = value
```

Why learn this?

Not because you'll implement NumPy in an interview.

Learn it to understand:

2-D abstraction

↓

1-D memory

↓

index transformation

↓

array access

That's the **engineering insight**.

---

32. NumPy Matrix

In practice:

```
A = np.array([
   [0, 1, 2, 3],
   [4, 5, 6, 7],
   [8, 9, 10, 11]
])
```

Then:

```
A.shape
```

returns:

(3,4)

You can access:

```
A[1,2]
```

→ 6

Column:

```
A[:,2]
```

Row:

A[1,:]

The chapter demonstrates these indexing and slicing operations.

---

33. Reshape ⭐ ⭐ ⭐ ⭐ ⭐

Definition

reshape() changes the shape of an array while preserving its elements.

Example:

x = np.array([1,2,3,4,5,6])

x.reshape(2,3)

Result:

[1 2 3]

[4 5 6]

Diagram

Original

[1 2 3 4 5 6]

|

| reshape(2,3)

▼

[1 2 3]

[4 5 6]

---

34. -1 in reshape ⭐ ⭐ ⭐ ⭐ ⭐

Very common interview/practical concept.

x.reshape(-1, 2)

means:

"NumPy, calculate the first dimension for me."

If there are 12 elements:

x.reshape(-1, 2)

becomes:

6 × 2

because:

6 × 2 = 12

The source specifically highlights -1 as a useful way to let NumPy infer an unknown dimension.

---

35. Reshape Does Not Change the Original Array

Example:

B = A.reshape(6,2)

The source notes that reshaping does not destructively change the original A.

Interview answer:

"reshape() changes how the array is viewed/shaped; it doesn't conceptually change the sequence of values."

---

36. Shape Is a Major Source of ML Bugs ⭐ ⭐ ⭐ ⭐ ⭐

Think of every tensor as:

data + shape

Example:

X.shape = (1000, 20)

means:

1000 samples

20 features

If:

W.shape = (20, 5)

then:

X @ W

gives:

(1000,20) @ (20,5)

↓

(1000,5)

Diagram:

1000 samples

|

▼

1000 × 20

|

| @

▼

20 × 5

|

▼

1000 × 5

This is one of the most useful ways to reason about ML code.

The source explicitly warns that shape mismatches are a major practical issue and that understanding shapes is vital.

---

37. Batch-First vs Batch-Last ⭐ ⭐ ⭐ ⭐

This is a subtle but important concept.

Batch-first

number of samples × number of features

Example:

(1000, 20)

means:

1000 samples

20 features

Batch-last

number of features × number of samples

Example:

(20, 1000)

The chapter uses batch-last in its mathematical treatment but notes that frameworks such as PyTorch and TensorFlow commonly use batch-first conventions.

Why this causes bugs

Imagine:

X = (1000,20)

but your algorithm expects:

(20,1000)

The code may fail immediately—or worse, broadcasting may make something execute incorrectly.

---

38. Column Vector Representation

Suppose:

x1 = np.array([2,0,0,0])

x2 = np.array([-1,1,0,0])

If these represent two samples as **columns**, use:

```
data = np.hstack([
    x1.reshape(-1,1),
    x2.reshape(-1,1)
])
```

Result:

[2 -1]

[0 1]

[0 0]

[0 0]

Shape:

(4,2)

The chapter demonstrates this exact issue and explains why explicit reshaping is necessary for 1-D arrays.

---

39. The Most Important Shape Diagram

Memorize this:

Individual sample:

x

shape = (features,)

Column representation:

x.reshape(-1, 1)

shape = (features, 1)

Multiple samples:

[x_1 x_2 x_3 ... x_n]

shape = (features, samples)

But in many practical ML frameworks:

[$sample_1$

$sample_2$

$sample_3$

...]

shape = (samples, features)

The source explicitly contrasts these two conventions.

---

40. Floating-Point Comparison ⭐ ⭐ ⭐

Another practical interview point.

Don't blindly do:

1.0 == 0.3 * 3 + 0.1

because floating-point representation can produce:

0.9999999999999999

rather than exactly:

1.0

The chapter demonstrates this issue directly.

Use:

np.allclose(a, b)

for numerical comparison.

Example:

```
np.allclose(
    1.0,
    0.3 * 3 + 0.1
)
```

---

41. Cosine Similarity ⭐ ⭐ ⭐ ⭐

The chapter's exercises ask you to implement cosine similarity.

Definition:

$$\boxed{\cos(x, y) = \frac{x \cdot y}{\| x \| \| y \|}}$$

Implementation

```
def cosine_similarity(x, y):
  return np.dot(x, y) / (
    np.linalg.norm(x) *
    np.linalg.norm(y)
  )
```

ML application

This is extremely important for:

Embedding A ──┐

├── cosine similarity ──> similarity score

Embedding B ──┘

Examples:

- semantic search
- document similarity
- recommendation
- duplicate detection
- RAG retrieval

The exercise is explicitly included in the source.

---

42. Hadamard Product

The Hadamard product is element-wise matrix multiplication.

$$\boxed{(A \odot B)_{ij} = A_{ij} B_{ij}}$$

Example:

A = [1 2]

[3 4]

B = [5 6]

[7 8]

A ⊙ B

= [5 12]

[21 32]

The chapter's exercises explicitly ask for both loop-based and NumPy implementations.

Implementation:

```
def hadamard(A, B):
    return A * B
```

---

43. 80/20 Interview Cheat Sheet

🔴 **Must memorize**

Vector

↓

array of numbers

Norm

↓

magnitude / length

Distance

↓

||x-y||

Dot product

↓

Σ xi yi

Matrix

↓

2-D numerical structure

Transpose

↓

rows ↔ columns

Matrix multiplication

↓

row × column dot products

Shape

↓

dimensions of array

Broadcasting

↓

automatic compatible dimension expansion

reshape

↓

change shape without changing element sequence

Ax

↓

linear transformation

A * B

↓

element-wise

A @ B

↓

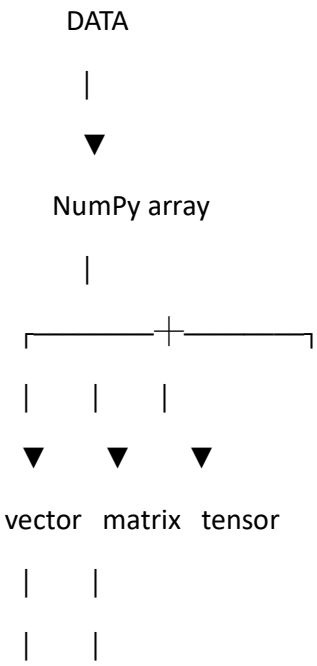
matrix multiplication

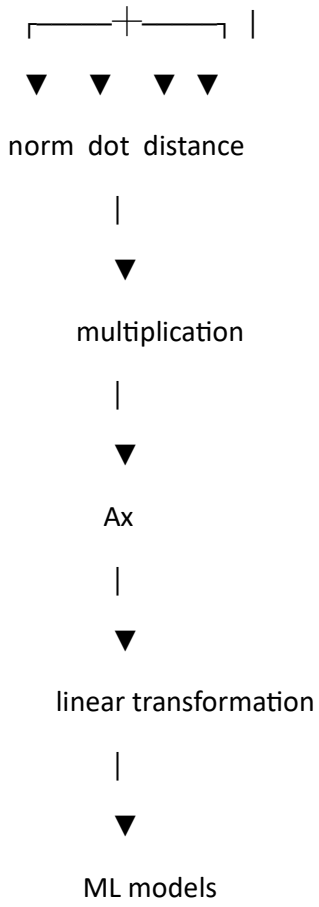
---

44. Interview-Level Mental Model

Don't memorize these as isolated NumPy APIs.

Think:





This is the conceptual chain you should understand.

---

45. Most Likely Interview Questions

Q1. What is vectorization?

Answer:

Vectorization means performing operations on an entire array rather than explicitly iterating over elements in Python. NumPy can execute these operations using optimized native numerical implementations.

Q2. What is a norm?

A norm measures the magnitude or length of a vector. The most common one is the L2/Euclidean norm.

$$\| x \|_2 = \sqrt{\sum_i x_i^2}$$

Q3. What is the dot product?

The dot product multiplies corresponding elements and sums them.

$$x \cdot y = \sum_i x_i \, y_i$$

Q4. What does a dot product tell us geometrically?

$$x \cdot y = \| x \| \| y \| \cos\theta$$

Therefore it gives information about the angle between vectors.

Q5. What is broadcasting?

Broadcasting allows NumPy to perform operations between arrays with compatible shapes by logically expanding dimensions.

Q6. Why can broadcasting be dangerous?

Because an unintended shape can still be broadcast successfully, producing a mathematically incorrect result without raising an exception.

This is specifically highlighted by the source.

Q7. Difference between * and @?

* → element-wise multiplication

@ → matrix multiplication

Q8. When can matrices be multiplied?

For:

$$A_{m \times n} B_{n \times p}$$

the inner dimensions must match.

Result:

$$m \times p$$

Q9. Is matrix multiplication commutative?

No.

$$AB \neq BA$$

in general.

Q10. What is transpose?

Transpose swaps rows and columns.

$$(A^T)_{ij} = A_{ji}$$

Q11. What is:

$$(AB)^T$$

?

$$\boxed{B^T A^T}$$

Q12. What is reshape(-1, 1)?

It converts a 1-D array into a column-shaped 2-D array.

x.shape

(4,)

x.reshape(-1, 1).shape

(4, 1)

Q13. What does Ax represent?

A matrix-vector multiplication that can be interpreted as a linear transformation.

Q14. What is Gram–Schmidt?

An algorithm that converts linearly independent vectors into an orthonormal set while preserving their span structure.

Q15. What can go wrong with Gram–Schmidt?

Linearly dependent or nearly dependent vectors can produce vectors with near-zero norm, causing numerical instability during normalization.

---

46. Production Debugging Checklist

When you get an ML shape bug, don't randomly change reshape().

Use this process:

1. Print shape

↓

2. Identify semantic meaning

↓

3. samples?

features?

channels?

sequence?

↓

4. Check operation

↓

5. Check expected output shape

↓

6. Check broadcasting

↓

7. Check transpose

↓

8. Check reshape

↓

9. Test with tiny data

Example:

print("X:", X.shape)

print("W:", W.shape)

print("b:", b.shape)

result = X @ W + b

print("result:", result.shape)

This is much more valuable in a real ML engineering interview than knowing 50 NumPy functions.

---

47. Your 80/20 Priority Map

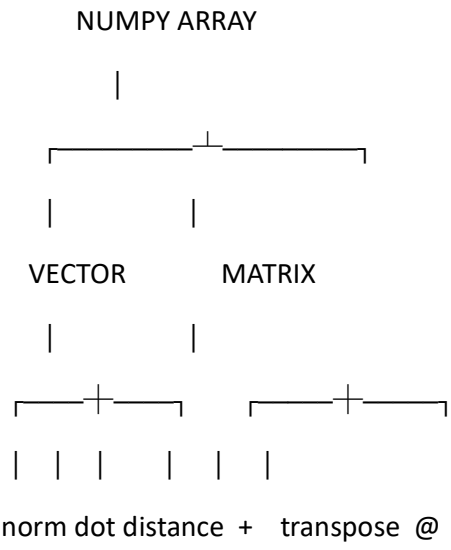
Concept	Priority	Interview Importance
NumPy arrays	🔴 P0	⭐⭐⭐⭐⭐
Shape	🔴 P0	⭐⭐⭐⭐⭐
Vectorization	🔴 P0	⭐⭐⭐⭐⭐
Broadcasting	🔴 P0	⭐⭐⭐⭐⭐
Dot product	🔴 P0	⭐⭐⭐⭐⭐
Norm	🔴 P0	⭐⭐⭐⭐
Distance	🔴 P0	⭐⭐⭐⭐
Matrix multiplication	🔴 P0	⭐⭐⭐⭐⭐
* vs @	🔴 P0	⭐⭐⭐⭐⭐

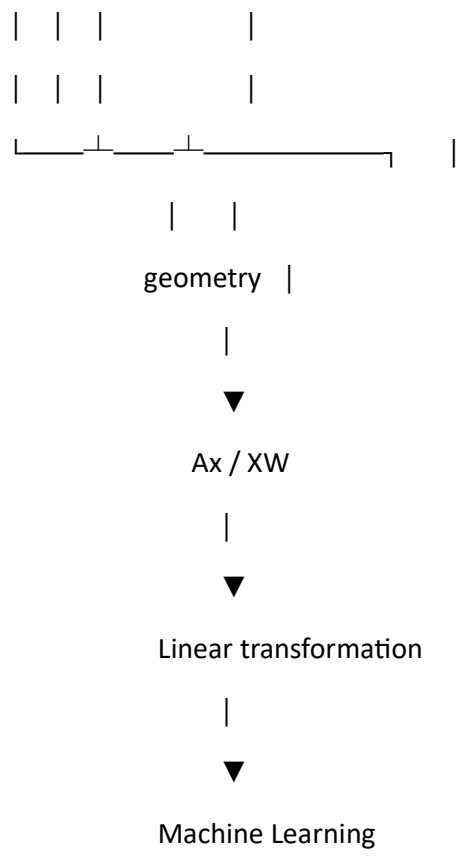
Concept	Priority	Interview Importance
Matrix × vector	🔴 P0	⭐⭐⭐⭐⭐
Transpose	🔴 P0	⭐⭐⭐⭐
Reshape	🔴 P0	⭐⭐⭐⭐⭐
Batch-first/last	🟠 P1	⭐⭐⭐⭐
Cosine similarity	🟠 P1	⭐⭐⭐⭐⭐
Projection	🟠 P1	⭐⭐⭐
Gram–Schmidt	🟠 P1	⭐⭐⭐
Numerical instability	🟠 P1	⭐⭐⭐⭐
Row-major storage	🟡 P2	⭐⭐⭐
Custom Matrix class	🟡 P2	⭐⭐
Hadamard product	🟡 P2	⭐⭐⭐
Bilinear form	🟡 P2	⭐⭐

---

48. Final Interview Mental Model

If you remember only one thing from this chapter, remember:





And for **every ML array problem**, ask:

"What does this shape mean?"

before asking:

"What NumPy function should I use?"

That mindset is exactly what the chapter is trying to develop: not merely knowing NumPy syntax, but understanding how vectors, matrices, shapes, transformations, and numerical computation fit together.

Next interview step

The uploaded chapter also contains **9 practice problems**, including MSE, performance comparison, p-norm, weighted p-norm, cosine similarity, matrix multiplication, Fibonacci/matrix computation, Hadamard product, and bilinear forms.

For your interview preparation, the best next step is to turn these into a **hands-on interview drill: question → think yourself → coding task → hidden edge case → expected answer → evaluation**, rather than just reading the solutions.